

Prompt 1: Apresentação para o Gemini:

Olá Gemini, estou graduando engenharia de software na faculdade CEUB de Brasília ,preciso que voce me ajude em um plano de implantação,seja um assistente meu e me guie.

Resposta:

Fala, futuro engenheiro! Parabéns pela escolha do curso. O CEUB tem uma tradição forte em Brasília, e a área de Engenharia de Software está voando. Vai ser um prazer ser o seu braço direito nessa missão.

Para estruturarmos um **Plano de Implantação** (Deployment Plan) robusto, digno de nota 10 na faculdade e de mercado no mundo real, nós precisamos seguir uma linha lógica. Basicamente, esse plano é o mapa de como o software sai do ambiente de desenvolvimento e chega com segurança e sucesso nas mãos do usuário final.

Para eu formatar o plano perfeito para o seu caso, vamos por partes. Um bom Plano de Implantação geralmente é dividido nestes pilares:

1. Escopo e Estratégia de Implantação

- **Qual será a estratégia?** (Ex: *Big Bang* – vira a chave de uma vez; *Faseada* – vai liberando aos poucos; ou *Shadow/Piloto* – roda em paralelo com o sistema antigo).
- **Janela de Manutenção:** Qual o melhor horário para rodar a implantação sem derrubar o negócio do cliente?

2. Infraestrutura e Pré-requisitos

- Onde o sistema vai rodar? (Local/On-premise, AWS, Azure, Google Cloud, Heroku, etc.).
- O que precisa estar instalado no servidor antes do deploy? (Bancos de dados, Docker, variáveis de ambiente).

3. Plano de Contingência (Rollback)

- **Deu ruim, e agora?** Se o sistema falhar no meio do processo, como desfazemos tudo rapidamente para o usuário não ficar fora do ar? (Backup de banco, espelhamento de versão anterior).

4. Cronograma de Execução (O "Passo a Passo")

- Lista detalhada de quem faz o quê, e em qual minuto da virada do sistema.

 **Dica de Engenharia:** No ambiente acadêmico, os professores adoram ver que você pensou nos riscos (o Plano de Rollback) e na automação (CI/CD), e não apenas em "subir o código".

Para a gente começar a desenhar o seu plano agora, me conta um pouco mais sobre o seu projeto:

Qual é o tipo de software que você está desenvolvendo e qual é a infraestrutura (nuvem, servidor local, etc.) que pretendem usar para a implantação?

Prompt 2: Deixando ciente ao Gemini o que eu já fiz

Fiz um portfolioHUB com currículo,apresentação em slide, site e um video no youtube,tambem criei um github pages para colocar essas coisas, agora preciso que ajude eu em uma nova implementação desse portfolioHUB ,a onde vou dizer oque deve ser seguido

Resposta:

Excelente! O projeto é um **PortfolioHUB**, centralizando currículo, slides, site e vídeo, e usando o **GitHub Pages** como infraestrutura principal. Essa é uma escolha muito inteligente e prática de

engenharia, pois o GitHub Pages é excelente para hospedar conteúdo estático de forma gratuita, rápida e com versionamento direto pelo Git.

Estou pronto. Como seu assistente, vou estruturar os passos e documentar essa nova implementação exatamente sob as suas ordens.

Pode mandar as diretrizes: **o que deve ser seguido e qual é a nova funcionalidade ou alteração que vamos implementar nesse PortfólioHUB?**

Prompt 3: Passo 1 pedido

Me explique como fazer isso na minha implementação:

- Criar e configurar um ambiente google workspace dedicado ao projeto
- Integrar as funcionalidades do google workspace no portfóliohub

Resposta:

Com certeza! Vamos estruturar essa nova implementação para o seu PortfólioHUB. Integrar o Google Workspace ao GitHub Pages vai transformar o seu portfólio estático em uma plataforma muito mais profissional e dinâmica, ideal para impressionar recrutadores e clientes.

Aqui está o guia passo a passo, estruturado como um verdadeiro plano de engenharia de software, para você seguir e documentar no seu projeto:

PARTE 1: Criar e Configurar o Ambiente Google Workspace Dedicado

Como você está usando o GitHub Pages, você provavelmente tem um domínio personalizado (ex: [seu-nome.com.br](#)) ou usa o padrão [seu-usuario.github.io](#). O ideal para o Workspace é ter um domínio próprio, pois ele permite criar e-mails profissionais (ex: [contato@seu-nome.com.br](#)).

Passo a Passo da Configuração:

1. Criação da Conta: Acesse o site do Google Workspace e faça o cadastro. Se você já tiver um domínio próprio configurado no GitHub Pages, insira-o durante o processo.
2. Verificação do Domínio (Crucial): O Google vai pedir para você provar que é dono do domínio. Como engenheiro, você fará isso adicionando um registro TXT na tabela DNS do provedor onde você comprou o seu domínio (Registro.br, GoDaddy, Hostgator, etc.).
3. Configuração de E-mail (Registros MX): Para que você possa receber e-mails no seu novo endereço profissional, você precisará alterar os registros MX (Mail Exchange) na mesma tabela DNS do seu domínio, apontando para os servidores do Google.
4. Organização do Drive Dedicado: Dentro do seu novo painel do Google Admin, crie uma pasta estruturada no Google Drive chamada [PortfólioHUB_Assets](#). É aqui que ficarão armazenados os arquivos que vamos integrar ao site.

PARTE 2: Integrar as Funcionalidades do Workspace no PortfólioHUB

O GitHub Pages aceita apenas arquivos estáticos (HTML, CSS, JavaScript). Portanto, para integrar o Google Workspace, nós temos duas abordagens principais: Embutir via Iframe (Sem código complexo) ou Consumir via API (Usando JavaScript).

Vamos detalhar as integrações para cada item do seu HUB:

1. Integração dos Slides (Google Apresentações)

Em vez de upar um PDF estático, você pode fazer com que seu slide atualize automaticamente no site sempre que você mexer nele no Workspace.

- No Google Slides: Abra sua apresentação → Vá em [Arquivo](#) → [Compartilhar](#) → [Publicar na Web](#).
- Escolha a aba Incorporar (Embed), configure o tamanho e clique em Publicar.
- No código do seu site (GitHub Pages): Copie o código `<iframe ...></iframe>` gerado e cole na seção correspondente do seu arquivo HTML.

2. Integração do Currículo (Google Docs)

Você pode permitir que as pessoas leiam seu currículo atualizado direto na tela ou façam o download.

- Opção Visualização (Iframe): Faça o mesmo processo do slide ([Arquivo](#) → [Compartilhar](#) → [Publicar na Web](#) → [Incorporar](#)) e cole o iframe no HTML.

- **Opção Botão de Download Direto:** No Google Docs, clique em [Compartilhar](#) e mude o acesso geral para *"Qualquer pessoa com o link pode ler"*. Copie o link de compartilhamento.
 - No código do seu site, transforme o link mudando o final dele. Se o link termina em [/edit?usp=sharing](#), mude para [/export?format=pdf](#). Isso criará um botão de download automático do seu currículo em PDF!

3. Captação de Contatos / Feedback (Google Forms)

Para tornar seu portfólio interativo, adicione um formulário para que recrutadores entrem em contato.

- No Google Forms: Crie o formulário de contato (Nome, E-mail, Mensagem).
- Clique em [Enviar](#), selecione o ícone de código (<>), Ajuste a largura e altura e copie o código do iframe.
- No seu código: Cole na seção de "Contato" do seu HTML. Todas as mensagens preenchidas no seu site vão cair direto em uma planilha do Google Sheets no seu Workspace.

4. Integração Avançada via API (Opcional - Diferencial de Engenharia)

Se você quiser mostrar que domina desenvolvimento frontend avançado, em vez de usar iframes (que às vezes quebram o design responsivo), você pode:

- Ir no Google Cloud Console do seu Workspace.
- Ativar a Google Drive API.
- Criar uma chave de API (API Key).
- Usar JavaScript ([fetch](#)) no seu código do GitHub Pages para buscar dinamicamente arquivos ou informações da sua conta Workspace e renderizá-los com o CSS nativo do seu site.

Próximos Passos do Nosso Plano:

Como seu assistente, recomendo começarmos pela Parte 1 (Configuração do Ambiente) ou pela forma estática dos Iframes (Parte 2).

Você já possui um domínio personalizado rodando no seu GitHub Pages ou está usando o endereço padrão [.github.io](#)? Me avise para eu detalhar exatamente como você deve prosseguir com os registros!

Prompt 4: Passo 2 pedido

Me explique como fazer isso na minha implementação:

3. Gestão de Usuários e Segurança:

- Configurar a gestão de usuários no Github e integrar com o PortfolioHUB;
- Implementar políticas de segurança, utilizando o Google GEMINI para garantir conformidade com as melhores práticas.

Resposta:

Excelente! Avançamos para um pilar fundamental da Engenharia de Software: **Segurança e Gestão de Identidade/Acesso (IAM)**.

No contexto de uma aplicação estática hospedada no **GitHub Pages**, o desafio de implementar controle de usuários e segurança é muito interessante, pois não temos um servidor back-end tradicional rodando 24h. Precisamos usar arquiteturas modernas (como Serverless ou serviços de terceiros).

Aqui está o plano de implementação detalhado para esses dois tópicos, pronto para você seguir e incluir na documentação do seu projeto:

ITEM 1: Gestão de Usuários no GitHub e Integração com o PortfolioHUB

Como o seu site está no GitHub Pages, existem duas formas de interpretar e implementar a "Gestão de Usuários":

Abordagem A: Gestão de Usuários para Colaboradores (Quem desenvolve o projeto)

Se o objetivo é gerenciar quem pode mexer no código ou aprovar alterações no PortfolioHUB:

1. **Configuração do Repositório:** Vá nas configurações ([Settings](#)) do seu repositório no GitHub.
2. **Collaborators:** Adicione os e-mails ou usuários do GitHub de outras pessoas que trabalham com você.
3. **Protected Branches (Ramas Protegidas):** Configure uma regra para a branch [main](#). Impeça que qualquer pessoa faça um [push](#) direto. Exija que seja feito um **Pull Request (PR)** e que pelo menos uma pessoa revise o código antes de ir para o ar.

Abordagem B: Autenticação de Usuários no Site (Quem acessa o PortfolioHUB) - O mais provável para o seu projeto

Se você quer que clientes, recrutadores ou você mesmo façam "Login" no seu PortfolioHUB usando a conta do GitHub (OAuth), o fluxo funciona assim:

Como o GitHub Pages é estático, você não pode esconder uma "Client Secret" de forma segura diretamente no JavaScript do navegador. A melhor prática de engenharia aqui é usar uma arquitetura **Jamstack** com um serviço auxiliar gratuito:

1. **Criar um OAuth App no GitHub:**
 - Vá no seu perfil do GitHub $\rightarrow$ **Settings** $\rightarrow$ **Developer Settings** $\rightarrow$ **OAuth Apps** $\rightarrow$ **New OAuth App**.
 - Configure a **Homepage URL** (a URL do seu GitHub Pages) e a **Authorization callback URL** (para onde o usuário volta após o login).
 - O GitHub vai te dar um **Client ID**.
2. **Utilizar um serviço de autenticação Serverless (Ex: Firebase Auth ou Supabase):**
 - Crie uma conta gratuita no **Supabase** ou **Firebase**.
 - Ative a autenticação via GitHub neles, colando o **Client ID** e a **Client Secret** fornecidos pelo GitHub.
3. **No código do seu PortfolioHUB:**
 - Instale ou importe o SDK do Supabase/Firebase via CDN no seu arquivo JavaScript.
 - Crie um botão "Login com GitHub" que chama a função de autenticação do serviço.
 - Pronto! O usuário se autentica pelo GitHub, e o serviço gerencia a sessão de forma segura diretamente no seu frontend estático.

ITEM 2: Implementar Políticas de Segurança e Uso do Google GEMINI para Conformidade

Para garantir que seu código e sua infraestrutura sigam as melhores práticas do mercado (como a cartilha da OWASP), nós vamos usar o **Google Gemini** como um consultor automatizado de segurança (DevSecOps). Aqui está o roteiro de como você vai aplicar o Gemini no ciclo de vida do seu PortfolioHUB:

1. Auditoria de Código e Prevenção de Vazamento de Credenciais (Hardcoded Secrets)

O erro mais comum de estudantes é deixar chaves de API, senhas ou tokens expostos no código do GitHub.

- **Ação com o Gemini:** Copie trechos do seu arquivo JavaScript principal ou de configuração e envie para o Gemini com o prompt:
- *"Atue como um engenheiro de segurança sênior. Analise este código JavaScript de um site estático hospedado no GitHub Pages e identifique se há alguma vulnerabilidade de segurança, exposição de chaves privadas (secrets) ou riscos de Cross-Site Scripting (XSS)."*
- **Implementação:** Siga as recomendações geradas. Se você tiver chaves que não podem ser expostas, utilize variáveis de ambiente nas **GitHub Actions** durante o processo de build, ou mova a lógica sensível para funções Serverless.

2. Criação de Políticas de Segurança (Security Content Policy - CSP)

A política CSP diz ao navegador quais fontes de scripts, imagens e estilos o seu site tem permissão para carregar. Isso evita que hackers injetem códigos maliciosos no seu portfólio.

- **Ação com o Gemini:** Forneça a lista de ferramentas que seu site usa (Google Workspace, YouTube, GitHub, Google Fontes) e peça:
- *"Gere uma tag HTML de Content Security Policy (CSP) robusta e segura para um site hospedado no GitHub Pages que consome recursos via iframe do Google Workspace (Docs, Slides, Forms) e vídeos do YouTube."*
- **Implementação:** O Gemini vai gerar uma tag `<meta http-equiv="Content-Security-Policy" content="...">`. Você deve colar essa tag exatamente dentro do `<head>` do seu `index.html`.

3. Configuração de Dependências Seguras no GitHub

- Ative o **Dependabot** nas configurações do seu repositório no GitHub. Ele varre seu projeto atrás de bibliotecas JavaScript desatualizadas ou com falhas de segurança conhecidas.
- Sempre que o Dependabot encontrar um problema, você pode colar o relatório da falha no Gemini e pedir:
- *"Como posso mitigar essa vulnerabilidade encontrada na dependência X sem quebrar a compatibilidade do meu projeto frontend?"*

Resumo para o seu Relatório da Faculdade:

Ao apresentar essa parte, destaque para os professores que você adotou o conceito de **DevSecOps** (Segurança integrada ao desenvolvimento):

- Usou **OAuth com provedores confiáveis** (GitHub + Supabase/Firebase) para não expor dados de usuários.
- Implementou **Políticas de CSP** no cabeçalho do site para mitigar ataques de injeção.
- Utilizou **Inteligência Artificial (Gemini)** como ferramenta de code-review focada em segurança e conformidade antes de realizar o deploy no GitHub Pages.

Qual dessas duas abordagens de gerenciamento de usuários (só para você controlar o código ou para pessoas fazerem login no site) se encaixa melhor no escopo que o seu professor pediu?

Prompt 4: Passo 3 pedido

Me explique como fazer essa proxima parte na minha implementacao:

4. Compartilhamento e Controle de Acesso com GitHub:

- Integrar o repositório ao PortfolioHUB, permitindo o controle de versão e o compartilhamento de código;
- Documentar o processo de configuração e práticas de colaboração.

Resposta:

Excelente escolha de tópicos. No desenvolvimento moderno de software, o controle de versão e a documentação de colaboração são o coração do trabalho em equipe. Para a banca da faculdade e para o seu portfólio profissional, dominar essa engrenagem demonstra maturidade técnica.

Como o GitHub Pages funciona lendo diretamente um repositório do GitHub, a integração nativa já existe, mas o segredo de engenharia está em como você estrutura esse fluxo de trabalho (Workflow) e como documenta isso. Aqui está o guia de implementação e documentação pronto para você aplicar e inserir no seu plano:

ITEM 1: Integrar o Repositório ao PortfolioHUB e Controle de Versão

Para garantir que cada alteração no código seja rastreada e que o compartilhamento com outros desenvolvedores seja seguro, siga este fluxo técnico:

1. Estrutura de Branches (Ramificações)

Nunca faça alterações diretamente na branch principal (**main** ou **master**) em produção. Adote um modelo simplificado do *Git Flow*:

- **main**: Armazena o código estável e homologado. É a branch que o GitHub Pages vai ler para atualizar o site público.
- **develop**: Onde os recursos novos são integrados e testados antes de irem para o ar.
- **feature/nome-da-feature**: Branches temporárias criadas a partir da **develop** para trabalhar em tarefas específicas (Ex: **feature/integracao-google-workspace**).

2. Automação de Deploy com GitHub Actions (CI/CD)

Em vez de depender do deploy padrão automático do GitHub Pages, configure uma Action para garantir a qualidade do código antes dele ser publicado.

- No seu repositório, crie a pasta **.github/workflows/** e adicione um arquivo chamado **deploy.yml**.
- Estructure o workflow para que, sempre que houver um **push** ou **merge** aprovado na branch **main**, o GitHub execute testes estáticos (linters) e, se tudo passar, publique no Pages.

ITEM 2: Documentar o Processo de Configuração e Práticas de Colaboração

Uma boa documentação é o que separa um programador amador de um Engenheiro de Software. Toda essa lógica deve ficar centralizada no arquivo **README.md** na raiz do seu repositório.

Aqui está o modelo exato de texto (padrão de mercado) que você deve colocar no seu **README.md** para documentar essas práticas:

Markdown

🚀 PortfolioHUB - Guia de Configuração e Colaboração

Este documento orienta os desenvolvedores sobre como configurar o ambiente local e seguir as melhores práticas de engenharia e controle de versão do projeto.

🚀 1. Configuração do Ambiente Local

Para clonar o projeto e rodar na sua máquina, siga os passos abaixo:

```
```bash
1. Clone o repositório
git clone https://github.com/seu-usuario/portfolioHUB.git

2. Navegue até o diretório do projeto
cd portfolioHUB

3. Se houver dependências (ex: pacotes npm), instale-as
npm install

4. Abra o projeto no seu editor (Ex: VS Code)
code .
```

## 🌿 2. Política de Branching (Git Flow Simplificado)

Para manter a integridade do código em produção, adotamos a seguinte estrutura:

- **main**: Versão estável em produção (refletida no GitHub Pages). *Bloqueada para commits diretos.*
- **develop**: Ambiente de integração para testes.
- **feature/\***: Branches para novas funcionalidades (Ex: **feature/ajuste-css**).

## 🤝 3. Fluxo de Colaboração (Como Contribuir)

Nenhum código vai direto para a **main**. Todo desenvolvedor deve seguir este processo:

1. A partir da branch **develop**, crie sua branch de feature:

Bash

```
git checkout -b feature/nome-da-sua-feature
...

2. Faça suas alterações e realize commits semânticos (padrão *Conventional Commits*):
```bash
git commit -m "feat: adicionado iframe do Google Slides"
...

3. Envie a branch para o GitHub:
```bash
git push origin feature/nome-da-sua-feature
...

4. Abra um **Pull Request (PR)** apontando para a branch `develop`.
5. **Revisão de Código (Code Review)**: Pelo menos um outro membro do projeto deve revisar as alterações e garantir conformidade com as diretrizes de segurança geradas pelo Google Gemini antes do *Merge*.
```

## ## 📝 4. Padrão de Commits

Para manter o histórico legível, utilize os seguintes prefixos nos commits:

- \* **feat:** Nova funcionalidade ou recurso.
- \* **fix:** Correção de um bug ou problema.
- \* **docs:** Alterações exclusivamente na documentação (como este README).
- \* **style:** Mudanças de formatação ou estilo que não afetam a lógica (CSS, espaçamento).

## 💡 Dica de Engenharia para Impressionar a Banca:

Vá nas configurações do seu repositório no GitHub (Settings -> Branches), clique em Add branch protection rule para a branch **main**, e marque a opção "Require a pull request before merging".

**Isso vai travar o repositório para que ninguém consiga quebrar o site sem querer, forçando o fluxo que você acabou de documentar no **README.md**.**



O que achou dessa estrutura para o seu portfólio? Quer que eu detalhe mais alguma regra específica de automação de testes antes do deploy?

#### Prompt 4: Prompt de dúvida

O que é essa branch protection rule para a branch main

#### Resposta:

A Branch Protection Rule (Regra de Proteção de Branch) é uma funcionalidade de segurança e governança do GitHub. Na Engenharia de Software, ela funciona como uma "trava de segurança" para garantir que ninguém — nem mesmo você, por acidente — envie um código quebrado ou sem testar direto para a versão pública do sistema (que, no seu caso, está rodando no GitHub Pages através da branch `main`).

Imagine a branch `main` como a linha de produção final de uma fábrica. Sem proteção, qualquer funcionário pode jogar uma peça defeituosa lá dentro. Com a regra de proteção, a peça precisa passar obrigatoriamente por uma esteira de inspeção antes de entrar na linha.

#### Por que usar na branch `main` do PortfolioHUB?

Como o seu site atualiza automaticamente assim que um código novo entra na `main`, se você cometer um erro de digitação no HTML ou esquecer uma tag aberta, o seu site sairá do ar imediatamente para qualquer recrutador que tentar acessá-lo.

A Branch Protection impede isso.

#### As 3 principais travas que você pode ativar:

Quando você cria essa regra para a `main`, você geralmente ativa três opções principais:

1. **Require a pull request before merging (Exigir um Pull Request antes do Merge):**
  - O que faz: Ninguém consegue dar um `git push origin main` direto do computador. Se tentar, o GitHub bloqueia.
  - Como funciona: Você é obrigado a subir seu código para uma branch separada (ex: `feature/novo-slide`), abrir um Pull Request (PR) no site do GitHub, e só depois "juntar" (fazer o merge) na `main`.
2. **Require approvals (Exigir aprovações):**
  - O que faz: Você pode configurar para que o código só entre na `main` se outra pessoa do seu grupo da faculdade (ou um revisor) olhar o código e clicar em "Aprovar". É o famoso *Code Review*.
3. **Require status checks to pass before merging (Exigir testes de validação):**
  - O que faz: Se você configurou testes automatizados (como um validador de HTML/CSS ou o Dependabot de segurança), o GitHub só deixa o código entrar na `main` se todos os testes ficarem verdes . Se o teste apontar um erro, o botão de publicar fica cinza e bloqueado.

#### Como isso agrega valor ao seu projeto no CEUB?

Para os seus professores de engenharia, apresentar um projeto com Branch Protection configurada mostra que você não está apenas "fazendo um site", mas sim gerenciando o ciclo de vida de um software com maturidade de mercado. Mostra que você entende o conceito de *Continuous Delivery* (Entrega Contínua) com foco em qualidade e redução de riscos de ambiente.

#### Melhorias de implementação comentadas no vídeo:

- Captação de Contatos / Feedback (Google Forms)
- Prevenção de Vazamento de Credenciais (Enviar códigos para Gemini conferir)
- Configuração de Dependências Seguras no GitHub